



**INSTITUTO
FEDERAL**
Santa Catarina

Visões (Views), Procedimentos Armazenados (Procedures) e Gatilhos (Triggers)

Instituto Federal de Santa Catarina

Análise e Desenvolvimento de Sistemas
Banco de dados

Prof. Ana Luiza Scharf
ana.scharf@ifsc.edu.br



INSTITUTO
FEDERAL
Santa Catarina

O que é uma *View*?

O que é uma *View*?

- Uma **View** (ou visão) é uma **tabela virtual**, baseada em uma ou mais tabelas reais, construída a partir de um **comando SELECT**.

O que é uma *View*?

- Uma **View** (ou visão) é uma **tabela virtual**, baseada em uma ou mais tabelas reais, construída a partir de um **comando SELECT**.
- **Não armazena** dados fisicamente, apenas exibe os dados de forma dinâmica, sempre atualizados com base nas tabelas originais.

Funcionamento de uma *View*?

- O SGBD armazena apenas a definição da **View** (nome e comando SELECT associado).

Funcionamento de uma *View*?

- O SGBD armazena apenas a definição da **View** (nome e comando SELECT associado).
- Quando a View é chamada, o banco executa automaticamente a consulta e retorna os **dados atualizados**.

Funcionamento de uma *View*?

- O SGBD armazena apenas a definição da **View** (nome e comando SELECT associado).
- Quando a View é chamada, o banco executa automaticamente a consulta e retorna os **dados atualizados**.
- Serve como uma **consulta pré-definida e reutilizável**, funcionando como se fosse uma **tabela temporária e dinâmica**.

Finalidade de uma *View*?

- **Simplificação** de consultas recorrentes e complexas.

Finalidade de uma *View*?

- **Simplificação** de consultas recorrentes e complexas.
- **Organização e padronização** do acesso aos dados.

Finalidade de uma *View*?

- **Simplificação** de consultas recorrentes e complexas.
- **Organização e padronização** do acesso aos dados.
- **Restrição de acesso a determinados dados**, funcionando como um mecanismo de segurança.

Finalidade de uma *View*?

- **Simplificação** de consultas recorrentes e complexas.
- **Organização e padronização** do acesso aos dados.
- **Restrição de acesso a determinados dados**, funcionando como um mecanismo de segurança.
- **Abstração da estrutura física do banco de dados**, promovendo independência dos dados.

Finalidade de uma *View*?

- **Simplificação** de consultas recorrentes e complexas.
- **Organização e padronização** do acesso aos dados.
- **Restrição de acesso a determinados dados**, funcionando como um mecanismo de segurança.
- **Abstração da estrutura física do banco de dados**, promovendo independência dos dados.
- **Facilita a manutenção e a gestão** da base de dados.



INSTITUTO
FEDERAL
Santa Catarina

Características de uma *View*?

- Estrutura virtual **baseada** em uma **consulta SQL** (SELECT).

Características de uma *View*?

- Estrutura virtual baseada em uma consulta SQL (SELECT).
- **Não ocupa espaço físico** com dados, apenas com sua definição.

Características de uma *View*?

- Estrutura virtual **baseada** em uma **consulta SQL** (SELECT).
- Não ocupa espaço físico com dados, apenas com sua definição.
- **Dados exibidos são atualizados em tempo real**, conforme o conteúdo das tabelas de origem.

Características de uma *View*?

- Estrutura virtual **baseada** em uma **consulta SQL** (SELECT).
- Não ocupa espaço físico com dados, apenas com sua definição.
- Dados exibidos são atualizados em tempo real, conforme o conteúdo das tabelas de origem.
- Permite **ocultar** colunas, linhas ou junções, conforme a necessidade de segurança ou simplicidade.



INSTITUTO
FEDERAL
Santa Catarina

Sintaxe de *Views*

```
CREATE VIEW view_name AS  
SELECT column1, column2, ...  
FROM table_name  
WHERE condition (...);
```

Exemplo de *Views*

- Criação de uma tabela de exemplo

```
CREATE DATABASE empresa;  
USE empresa;
```

```
CREATE TABLE funcionarios (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(100),  
  cargo VARCHAR(50),  
  departamento VARCHAR(50),  
  salario DECIMAL(10, 2),  
  status VARCHAR(20) );  
);
```

Exemplo de *Views*

- Inserção de dados na tabela

```
INSERT INTO funcionarios (nome, cargo, departamento, salario, status) VALUES  
('Ana Silva', 'Analista', 'TI', 5500.00, 'Ativo'),  
('Bruno Costa', 'Gerente', 'Financeiro', 8500.00, 'Ativo'),  
('Carlos Lima', 'Assistente', 'RH', 3200.00, 'Inativo'),  
('Débora Souza', 'Analista', 'TI', 6000.00, 'Ativo'),  
('Eduardo Pereira', 'Diretor', 'Administrativo', 15000.00, 'Ativo');
```

id	nome	cargo	departament o	salario	statu s
1	Ana Silva	Analista	TI	5500.00	Ativo
2	Bruno Costa	Gerente	Financeiro	8500.00	Ativo
3	Carlos Lima	Assiste nte	RH	3200.00	Inativ o
4	Débora Souza	Analista	TI	6000.00	Ativo
5	Eduardo Pereira	Diretor	Administrati vo	15000.0 0	Ativo

Exemplo de *Views*

- Criação de uma View para exibir apenas funcionários ativos

```
CREATE VIEW funcionarios_ativos AS  
SELECT id, nome, cargo, departamento, salario  
FROM funcionarios  
WHERE status = 'Ativo';
```

Exemplo de *Views*

- Consulta utilizando a View

```
SELECT * FROM funcionarios_ativos;
```



Exemplo de *Views*

id	nome	cargo	departamento	salario
1	Ana Silva	Analista	TI	5500.00
2	Bruno Costa	Gerente	Financeiro	8500.00
4	Débora Souza	Analista	TI	6000.00
5	Eduardo Pereira	Diretor	Administrativo	15000.00

Exemplo de *Views*

- Consulta utilizando a View

```
UPDATE funcionarios  
SET status = 'Inativo'  
WHERE nome = 'Débora Souza';  
  
SELECT * FROM funcionarios_ativos;
```


Exemplo de *Views*

id	nome	cargo	departamento	salario
1	Ana Silva	Analista	TI	5500.00
2	Bruno Costa	Gerente	Financeiro	8500.00
5	Eduardo Pereira	Diretor	Administrativo	15000.00

Exemplo de *Views*

- Exclusão da View

```
DROP VIEW funcionarios_ativos;
```

O que é uma *Procedures*?

Uma **Procedure** (Procedimento Armazenado) é um **bloco de comandos SQL** que fica salvo dentro do banco de dados e pode ser executado sob demanda. Ela funciona como **uma função** programada dentro do banco, permitindo agrupar operações que podem ser reutilizadas e otimizadas.

O que é um *Procedure*?

- Procedimento armazenado no banco de dados.

O que é um *Procedure*?

- Procedimento armazenado no banco de dados.
- Conjunto de comandos SQL que podem ser executados como uma rotina.

O que é um *Procedure*?

- Procedimento armazenado no banco de dados.
- Conjunto de comandos SQL que podem ser executados como uma rotina.
- Pode receber parâmetros de entrada e saída.

O que é um *Procedure*?

- Procedimento armazenado no banco de dados.
- Conjunto de comandos SQL que podem ser executados como uma rotina.
- Pode receber parâmetros de entrada e saída.
- Centraliza regras, melhora performance e segurança.

Vantagens *Procedure*

Performance: executa direto no servidor.

Segurança: restringe acesso direto às tabelas.

Padronização: centraliza regras de negócio.

Reutilização: evita repetir códigos SQL nas aplicações.

Manutenção fácil: alterações feitas direto na procedure.

Tipo de *Procedure*

Local: Criada no banco do usuário.

Temporária: Existe enquanto durar a sessão.

De Sistema: Procedures internas do próprio SGBD.

Remota: Executa em banco remoto (via Linked Server).

Estendida: Usa arquivos externos (.dll no SQL Server).

Procedimento Local (Padrão)

DELIMITER //

CREATE PROCEDURE listar_funcionarios()

BEGIN

SELECT * **FROM** funcionarios;

END //

DELIMITER ;

// Executando:

CALL listar_funcionarios();

Procedimento Local (Padrão)

id	nome	cargo	departamento	salario
1	Ana Silva	Analista	TI	5500.00
2	Bruno Costa	Gerente	Financeiro	8500.00
3	Carlos Lima	Assistente	RH	3200.00
4	Débora Souza	Analista	TI	6000.00
5	Eduardo Pereira	Diretor	Administrativo	15000.00

Obs: DELIMITER

- O **DELIMITER** não faz parte da procedure.
- Serve para o terminal entender que o ponto e vírgula ; interno não finaliza o bloco.
- Após a procedure, sempre voltamos para o delimitador padrão ;.

Parâmetro de entrada

```
DELIMITER //
```

```
CREATE PROCEDURE contar_funcionarios(IN dep VARCHAR(50))  
BEGIN  
    SELECT * FROM funcionarios  
    WHERE departamento = dep;  
END //
```

```
DELIMITER ;
```

```
// Executando:
```

```
CALL contar_funcionarios('TI');
```

Parâmetro de entrada

id	nome	cargo	departamento	salario
1	Ana Silva	Analista	TI	5500.00
4	Débora Souza	Analista	TI	6000.00

Parâmetro de saída

DELIMITER //

```
CREATE PROCEDURE buscar_por_departamento (  
    IN dep VARCHAR(50),  
    OUT total INT  
)  
BEGIN  
    SELECT COUNT(*) INTO total  
    FROM funcionarios  
    WHERE departamento = dep;  
END //
```

DELIMITER ;

// Executando:

```
CALL buscar_por_departamento ('TI', @total);  
SELECT @total;
```



INSTITUTO
FEDERAL
Santa Catarina

Parâmetro de saída

@ total
2

DELIMITER //

```
CREATE PROCEDURE aplicar_desconto ( IN dep DECIMAL(10,2))  
BEGIN  
    IF valor > 100 THEN  
        SELECT valor * 0.9 AS valor_com_desconto;  
    ELSE  
        SELECT valor AS valor_sem_desconto;  
    END IF;  
  
END //
```

DELIMITER ;

// Executando:

CALL aplicar_desconto(150);



**INSTITUTO
FEDERAL**
Santa Catarina

Condições

valor_com_desconto
135.00

DELIMITER //

```
CREATE PROCEDURE verificar_departamento( IN dep VARCHAR(50))
BEGIN
  SELECT CASE dep
    WHEN 'TI' THEN 'Departamento de Tecnologia'
    WHEN 'RH' THEN 'Departamento de Recursos Humanos'
    WHEN 'Financeiro' THEN 'Departamento Financeiro'
    ELSE 'Outro Departamento'
  END AS descricao;
END //
```

DELIMITER ;

// Executando:

CALL verificar_departamento('TI');



**INSTITUTO
FEDERAL**
Santa Catarina

Case

descricao

Departamento de Tecnologia



INSTITUTO
FEDERAL
Santa Catarina

Exclusão de uma Procedure

```
DROP PROCEDURE listar_funcionarios;
```

Exemplo de *Procedures*

```
DELIMITER //
```

```
CREATE PROCEDURE listar_ativos()  
BEGIN  
  SELECT * FROM funcionarios  
  WHERE status = 'Ativo';  
END //
```

```
DELIMITER ;
```

O que é uma *Trigger*?

É um objeto do banco de dados que executa comandos SQL **automaticamente em resposta a eventos** específicos dentro de uma tabela.

O que é uma *Trigger*?

- Objeto do banco de dados que executa comandos SQL de forma **automática**.

O que é uma *Trigger*?

- Objeto do banco de dados que executa comandos SQL de forma **automática**.
- É **ativado** por operações de **INSERT**, **UPDATE** ou **DELETE**.

O que é uma *Trigger*?

- Objeto do banco de dados que executa comandos SQL de forma **automática**.
- É **ativado** por operações de **INSERT**, **UPDATE** ou **DELETE**.
- Executa ações **vinculadas** a uma **tabela**, sem necessidade de chamada manual.

Funcionamento de um *Trigger*.

- **Associado** a uma **tabela** específica.
- Executado **antes** (BEFORE) ou **depois** (AFTER) de uma **operação**.
- Atua linha a linha: executa o código para cada registro afetado.
- Ativado **automaticamente** ao ocorrer a operação definida.

Finalidade de um *Trigger*:

- Registrar alterações (auditoria).
- Aplicar regras de negócio.
- Impedir operações que não atendam critérios.
- Automatizar ações que devem ocorrer sempre que dados forem modificados.

Sintaxe de *Trigger*

```
CREATE TRIGGER nome_trigger  
BEFORE | AFTER INSERT | UPDATE | DELETE  
ON nome_tabela  
FOR EACH ROW  
BEGIN  
    -- comandos SQL  
END;
```

Executa antes ou depois de uma operação de modificação.



INSTITUTO
FEDERAL
Santa Catarina

Exemplo de *Trigger*

```
CREATE TABLE log_exclusoes (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100),  
  data_exclusao DATETIME  
);
```

Exemplo de *Trigger*

```
DELIMITER //  
  
CREATE TRIGGER log_exclusao_funcionario  
AFTER DELETE ON funcionarios  
FOR EACH ROW  
BEGIN  
    INSERT INTO log_exclusoes (nome, data_exclusao)  
    VALUES (OLD.nome, NOW());  
END; //  
  
DELIMITER ;
```

Registra no log o nome e a data de exclusão do funcionário.

Exemplo de *Trigger*

```
SELECT * FROM log_exclusoes;
```

```
DELETE FROM funcionarios  
WHERE nome = 'Carlos Lima';
```

```
SELECT * FROM log_exclusoes;
```

Registra no log o nome e a data de exclusão do funcionário.



INSTITUTO
FEDERAL
Santa Catarina

Exemplo de *Trigger*

id	nome	data_exclusao
1	Carlos Lima	2024-06-02 14:35:22

Exemplo de *Trigger*

```
DELIMITER //
```

```
CREATE TRIGGER validar_salario  
BEFORE INSERT ON funcionarios  
FOR EACH ROW  
BEGIN  
    IF NEW.salario <= 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Salário deve ser maior que zero';  
    END IF;  
END; //
```

```
DELIMITER ;
```

Exemplo de *Trigger*

- `SIGNAL SQLSTATE '45000'` → Código de erro genérico customizado.

Exemplo de *Trigger*

```
INSERT INTO funcionarios (nome, cargo, departamento, salario, status)  
VALUES ('Fernanda Alves', 'Assistente', 'RH', 3500.00, 'Ativo');
```

```
INSERT INTO funcionarios (nome, cargo, departamento, salario, status)  
VALUES ('João Mendes', 'Analista', 'TI', 0, 'Ativo');
```

```
INSERT INTO funcionarios (nome, cargo, departamento, salario, status)  
VALUES ('Maria Silva', 'Gerente', 'Financeiro', -5000, 'Ativo');
```

id	nome	cargo	departamen to	salario	statu s
1	Ana Silva	Analista	TI	5500.00	Ativo
2	Bruno Costa	Gerente	Financeiro	8500.00	Ativo
4	Débora Souza	Analista	TI	6000.00	Ativo
5	Eduardo Pereira	Diretor	Administrati vo	15000.0 0	Ativo
6	Fernanda Alves	Assistente	RH	3500.00	Ativo



INSTITUTO
FEDERAL
Santa Catarina

Remoção de um *Trigger*

```
DROP TRIGGER log_exclusao_funcionario;
```